

Python Functions, Recursion, Modules & Packages — Practical Learning Index

1. Function Basics

Theory titles

- What is a Function?
- Why Use Functions?
- Function Definition
- Function Calling
- Function Body
- `def` Keyword
- Function Naming
- Function Execution Flow
- Built-in vs User-defined Functions

Problems

1. Create a function that prints "Hello, World!".
 2. Create a function that prints your name.
 3. Create a function that prints your name, age, and university.
 4. Create a function to print numbers from 1 to 10.
 5. Create a function to print the multiplication table of 5.
 6. Create a function to check whether a number is even or odd.
 7. Create a function to calculate the square of a number.
 8. Create a function to calculate the area of a rectangle.
 9. Create a function that prints all numbers between two given values.
-

2. Parameters

Theory titles

- What is a Parameter?
- Parameter vs Argument
- Single Parameter
- Multiple Parameters
- Passing Values to Functions
- Positional Arguments
- Parameter Order
- Number of Arguments

Problems

1. Function that accepts a name and prints a greeting.
 2. Function that accepts two numbers and prints their sum.
 3. Function that accepts three numbers and prints their average.
 4. Function that accepts length and width and calculates area.
 5. Function that accepts a number and prints its multiplication table.
 6. Function that accepts a number and checks whether it is positive, negative, or zero.
 7. Function that accepts two numbers and prints the larger one.
 8. Function that accepts three numbers and prints the largest.
 9. Function that accepts a student's name and marks and prints the result.
-

3. Return Values

Theory titles

- `return` Statement
- Returning a Value
- `print()` vs `return`
- Storing Returned Values
- Returning Multiple Values
- Using Returned Values in Expressions
- Function Execution with `return`

Problems

1. Return the square of a number.
 2. Return the sum of two numbers.
 3. Return the largest of two numbers.
 4. Return whether a number is even.
 5. Return the area of a rectangle.
 6. Return the average of three numbers.
 7. Return the factorial of a number.
 8. Return the maximum and minimum of two numbers.
 9. Create a calculator function that returns the result.
 10. Create a function that returns a student's grade.
-

4. Default Arguments

Theory titles

- What is a Default Argument?
- Why Use Default Arguments?
- Default Parameter Syntax
- Default vs Required Parameters
- Multiple Default Arguments
- Parameter Order Rules

Problems

1. Create a greeting function with a default name.
 2. Create a function with a default country.
 3. Create a function that calculates price with a default tax rate.
 4. Create a function with a default discount.
 5. Create a function that calculates power with default exponent 2.
 6. Create a function that displays student information with optional values.
 7. Create a function where some arguments are required and others are optional.
-

5. Keyword Arguments

Theory titles

- What is a Keyword Argument?
- Positional vs Keyword Arguments
- Passing Arguments by Name
- Argument Order with Keyword Arguments
- Mixing Positional and Keyword Arguments
- Keyword Arguments with Default Values

Problems

1. Create a student information function and call it using keyword arguments.
 2. Create a rectangle function and pass `length` and `width` by name.
 3. Create a profile function with `name`, `age`, and `country`.
 4. Call the same function using different argument orders.
 5. Combine positional and keyword arguments.
 6. Create a function with default + keyword arguments.
-

6. Variable Scope

Theory titles

- What is Variable Scope?
- Local Variables
- Global Variables
- Local vs Global
- Accessing Global Variables
- `global` Keyword
- Scope Inside Functions
- Scope Outside Functions

Problems

1. Create a local variable inside a function.
 2. Try to access a local variable outside the function.
 3. Create a global variable and access it inside a function.
 4. Create a local variable with the same name as a global variable.
 5. Modify a global variable inside a function.
 6. Use the `global` keyword to update a counter.
 7. Create two functions that use variables with the same name and observe their scope.
 8. Build a simple program using global and local variables.
-

7. Lambda Functions

Theory titles

- What is a Lambda Function?
- Lambda Syntax
- Lambda Parameters
- Lambda Return Value
- Lambda vs Normal Function
- Lambda with One Parameter
- Lambda with Multiple Parameters
- Lambda with `if-else`
- Lambda with Built-in Functions

Problems

1. Create a lambda that squares a number.
 2. Create a lambda that adds two numbers.
 3. Create a lambda that finds the larger of two numbers.
 4. Create a lambda that checks whether a number is even.
 5. Create a lambda that converts Celsius to Fahrenheit.
 6. Create a lambda that calculates the area of a rectangle.
 7. Create a lambda that returns "Adult" or "Minor".
 8. Use lambda with `map()`.
 9. Use lambda with `filter()`.
 10. Use lambda with `sorted()`.
-

8. Recursion Basics

Theory titles

- What is Recursion?
- Recursive Function
- Base Case
- Recursive Case
- Function Calling Itself
- Recursion Flow
- Recursion vs Loop
- Infinite Recursion
- Recursion Depth

Problems — Basic

1. Print numbers from 1 to 10 using recursion.
2. Print numbers from 10 to 1 using recursion.
3. Calculate the sum from 1 to n .
4. Calculate the factorial of n .
5. Calculate x^n using recursion.
6. Find the n th Fibonacci number.
7. Count digits of a number recursively.
8. Find the sum of digits of a number.
9. Reverse a string using recursion.
10. Check whether a string is a palindrome using recursion.

Problems — Intermediate

11. Find the greatest common divisor using recursion.
 12. Convert decimal to binary using recursion.
 13. Find the maximum element in a list recursively.
 14. Count occurrences of an element in a list.
 15. Implement binary search recursively.
-

9. Modules

Theory titles

- What is a Module?
- Why Use Modules?
- Python `.py` Files as Modules
- Built-in Modules
- User-defined Modules
- Module Namespace
- Module Reusability

Problems

1. Import and use Python's `math` module.
 2. Use the `random` module to generate random numbers.
 3. Use the `datetime` module.
 4. Use the `os` module.
 5. Use the `statistics` module.
 6. Create a program that generates random passwords using modules.
 7. Create a calculator using functions stored in a module.
-

10. Import Statements

Theory titles

- `import module`
- `from module import`
- `import ... as`
- Importing Multiple Items
- Import Aliases
- Importing Specific Functions
- Importing Everything
- Module Namespace

Problems

1. Import `math` and calculate square root.
 2. Import only `sqrt` from `math`.
 3. Import `math` using an alias.
 4. Import multiple functions from a module.
 5. Create a program using `random.randint()`.
 6. Compare `import math` vs `from math import sqrt`.
 7. Create a program that uses multiple modules together.
-

11. Creating Custom Modules

Theory titles

- Creating Your Own .py Module
- Writing Functions Inside a Module
- Importing Your Custom Module
- Calling Custom Functions
- Module Variables
- Module Constants
- Module Reusability
- `if __name__ == "__main__":`

Problems

1. Create `calculator.py` containing:
 - `add()`
 - `subtract()`
 - `multiply()`
 - `divide()`
 2. Import `calculator.py` into another Python file.
 3. Create `math_utils.py` containing:
 - `square()`
 - `cube()`
 - `factorial()`
 4. Create `student.py` containing functions for:
 - calculating average
 - calculating grade
 - displaying student information
 5. Create `temperature.py` containing:
 - Celsius → Fahrenheit
 - Fahrenheit → Celsius
 6. Create `geometry.py` containing:
 - rectangle area
 - circle area
 - triangle area
 7. Create a custom module and import it using an alias.
 8. Create a project containing **3 custom modules** and one main Python file that uses all of them.
-

12. Packages

Theory titles

- What is a Package?
- Module vs Package
- Package Directory Structure
- `__init__.py`
- Importing from a Package
- Submodules
- Package Hierarchy

Problems

1. Create a package called `calculator`.
 2. Put separate modules inside it:
 - `addition.py`
 - `subtraction.py`
 - `multiplication.py`
 - `division.py`
 3. Import functions from those modules.
 4. Create a `student` package containing:
 - `marks.py`
 - `grade.py`
 - `profile.py`
 5. Create a `geometry` package containing:
 - `circle.py`
 - `rectangle.py`
 - `triangle.py`
 6. Create a project using **package** → **modules** → **functions**.
-

13. Final Practical Projects

After completing the individual problems, build these without looking at your notes.

Project 1 — Calculator

Use:

- Functions
- Parameters
- Return values
- Default arguments
- Modules

Project 2 — Student Management

Use:

- Functions
- Parameters
- Return values
- Scope
- Modules
- Packages

Project 3 — Utility Toolkit

Create your own package:

```
utility/  
├── calculator.py  
├── converter.py  
├── validator.py  
├── math_utils.py  
└── __init__.py
```

Then create:

```
main.py
```

and use the package.

Project 4 — Recursive Toolkit

Create functions for:

```
factorial()  
fibonacci()  
sum_digits()  
reverse_string()  
is_palindrome()  
binary_search()
```

Project 5 — Mini Python Application

Build a small application that combines:

```
Functions  
  ↓  
Parameters  
  ↓  
Return Values  
  ↓  
Default / Keyword Arguments  
  ↓  
Scope  
  ↓  
Lambda  
  ↓  
Recursion  
  ↓  
Modules  
  ↓  
Packages
```

Best Learning Order

Don't study everything separately for hours. Follow this sequence:

1. Function Basics
- ↓
2. Parameters
- ↓
3. Return Values
- ↓
4. Default Arguments
- ↓
5. Keyword Arguments
- ↓
6. Variable Scope
- ↓
7. Lambda
- ↓
8. Recursion
- ↓
9. Modules
- ↓
10. Import Statements
- ↓
11. Custom Modules
- ↓
12. Packages
- ↓
13. Projects

Rule: For each theory title, learn only enough syntax to solve the problems immediately. If you can solve the problems without looking at the answer, you've learned that topic practically.